Etude du retour à l'état initial de la disposition d'un jeu de carte

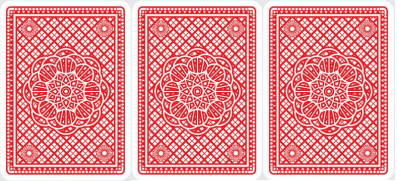
(à travers le mélange Riffle Shuffle)

CHAUBET MARTIN
40252

Principe du mélange









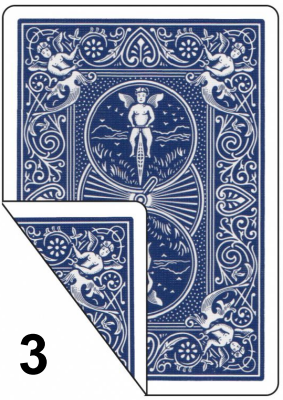
Sommaire

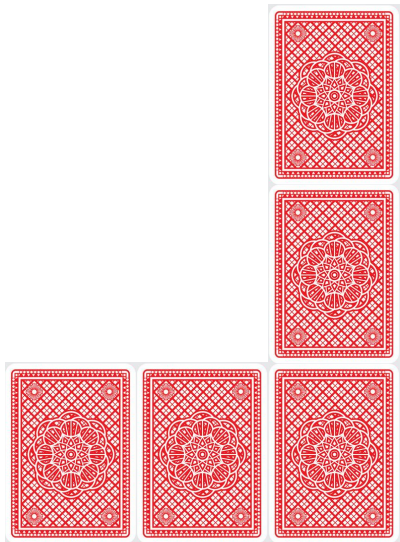
1) *Etude d'un mélange parfait (Perfect Shuffle)*

2) *Etude du mélange réel*

3) *Conclusion sur le retour à l'état initial*

4) *Annexes*





1) *Etude du Perfect Shuffle*
(mélange supposé parfait)

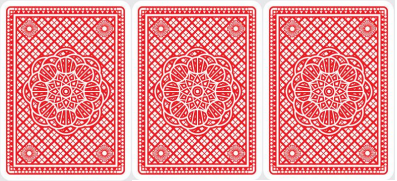
$\{1,2,3,...,2n-1,2n\}$ $n \in \mathbb{N}^*$

Coupe en deux

① $\{1,2,...,n\}$ $\{n+1,...,2n\}$ ②

Entrelacement

$\{1, n+1, 2, n+2, 3,...,n, 2n\}$





① Si $1 < i \leq n$: La carte va en position impaire, en $2i-1$

② Si $n+1 \leq i < 2n$: La carte va en position paire, on note $i=n+j$ avec $j \in \{1,...,n\}$ de sorte que i balaye les positions $(n+1,...,2n)$

Pour chaque carte du paquet ②: $j \rightarrow 2j = 2i-2n$ (position d'une carte du paquet 2 dans le paquet "réunifié")

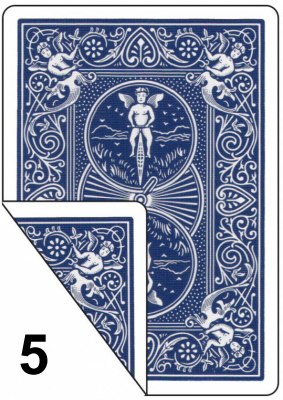
<u>Formule générale sur la position d'une carte dans le paquet</u>

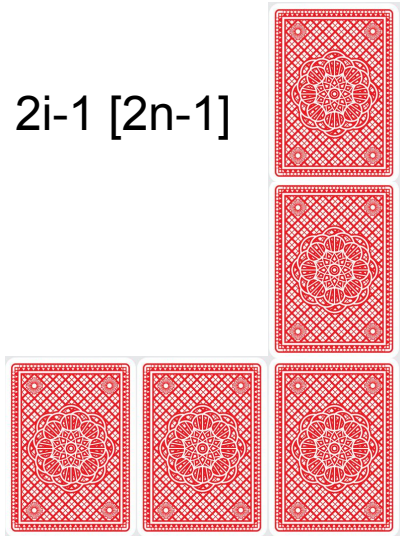
$$\begin{cases} ① \quad i \rightarrow 2i-1 \\ ② \quad i \rightarrow 2i-2n \end{cases}$$

or $(2i-1)-(2i-2n) = 2n-1$ donc $2i-2n \equiv 2i-1 \ [2n-1]$

:

D'où dans les 2 cas : $i \rightarrow 2i-1 \ [2n-1]$





1 mélange : $i \rightarrow 2i-1\ [2n-1]$

2 mélange : $i \rightarrow 2(2i-1)-1\ [2n-1]$

.

.

k mélanges : $i \rightarrow 2^k i-(2^k-1)\ [2n-1]$

$$i \rightarrow 2^k(i-1)+1\ [2n-1]$$

<u>Retour de la carte à la position i :</u>

$2^k(i-1)+1 \equiv i\ [2n-1] \quad \Leftrightarrow \quad (i-1)(2^k-1) \equiv 0\ [2n-1]$

Or ceci est vrai pour tout i ((i-1) prend les valeurs {0,...,2n-1})

D'où : $\boxed{2^k \equiv 1\ [2n-1]}$

Le plus petit entier k qui vérifie cette condition, correspond au nombre de mélanges avant retour à l'état initial

Avec une décomposition en cycle disjoint
Cas de 2n cartes :

(1, 2, 3, 4, ……. 2n) on mélange (1, n+1, 2, n+2, ……. n, 2n)

(2 5 3)(4 6 7)

(1, 2, 3, 4, 5, 6, 7, 8) → (1, 5, 2, 6, 3, 7, 4, 8)

(5 3 2)(6 7 4)

→ (1, 3, 5, 7, 2, 4, 6, 8)

(3 2 5)(7 4 6)

→ (1, 2, 3, 4, 5, 6, 7, 8)

Cas général :

L'exemple précédent : (2 5 3)(4 6 7)
Décomposition en cycle disjoint :

$\sigma = S_1 o S_2 o \ldots o S_r$

m_i la longueur du cycle S_i , $1 \leq i \leq r$

$\sigma^p = S_1^p \text{ o } S_2^p \text{ o} \ldots \text{o } S_r^p = Id$

$p = ppcm (m_1, m_2, \ldots, m_r)$

Cas de 2n+1 cartes :

(1, 2, 3, 4, 5, ……. 2n, 2n+1)

Paquet du bas : (1, 2, 3, ….. n)

Paquet du dessus : (n+1, n+2, …… 2n, 2n+1)

On mélange (1, n+1, 2, n+2, ……. n, 2n, 2n+1)

Paquet du bas : (1, 2, 3, ….. n, n+1)

Paquet du dessus : (n+2, …… 2n, 2n+1)

(1,n+2, 2, n+3, ……. n, 2n+1, n+1)

Quelques applications

Nombre de cartes	52	32	16
Nombre de mélanges	8	5	4

Nombre de cartes	52	32	16
Nombre de mélanges (obtenu avec formule)	$2^8 \equiv 1\ [51]$ $5 \times 51 + 1 = 256$	$2^5 \equiv 1\ [31]$ $31 + 1 = 32$	$2^4 \equiv 1\ [15]$ $15 + 1 = 16$

Confirmation par un outil numérique

Méthode ppcm :

```
>>> cycle_disjoint(L)
[[1, 4, 2], [3, 5, 6]]

>>> ppcm(L)
3

>>> cycle_disjoint([i for i in range(52)])
[[1, 26, 13, 32, 16, 8, 4, 2], [3, 27, 39, 45, 48, 24, 12, 6], [5,
28, 14, 7, 29, 40, 20, 10], [9, 30, 15, 33, 42, 21, 36, 18], [11, 3
1, 41, 46, 23, 37, 44, 22], [17, 34], [19, 35, 43, 47, 49, 50, 25,
38]]

>>> ppcm([i for i in range(52)])
8

>>> cycle_disjoint([i for i in range(54)])
[[1, 27, 40, 20, 10, 5, 29, 41, 47, 50, 25, 39, 46, 23, 38, 19, 36,
18, 9, 31, 42, 21, 37, 45, 49, 51, 52, 26, 13, 33, 43, 48, 24, 12,
6, 3, 28, 14, 7, 30, 15, 34, 17, 35, 44, 22, 11, 32, 16, 8, 4, 2]]

>>> ppcm([i for i in range(54)])
52
```

Confirmation par un outil numérique

Méthode instinctive :

```
>>> L=[i for i in range(8)]

>>> L
[0, 1, 2, 3, 4, 5, 6, 7]

>>> coupe_paire(L)
[0, 4, 1, 5, 2, 6, 3, 7]

>>> J=[i for i in range(9)]

>>> J
[0, 1, 2, 3, 4, 5, 6, 7, 8]

>>> coupe_impaire(J)
[0, 5, 1, 6, 2, 7, 3, 8, 4]
```

Confirmation par un outil numérique

Méthode instinctive :

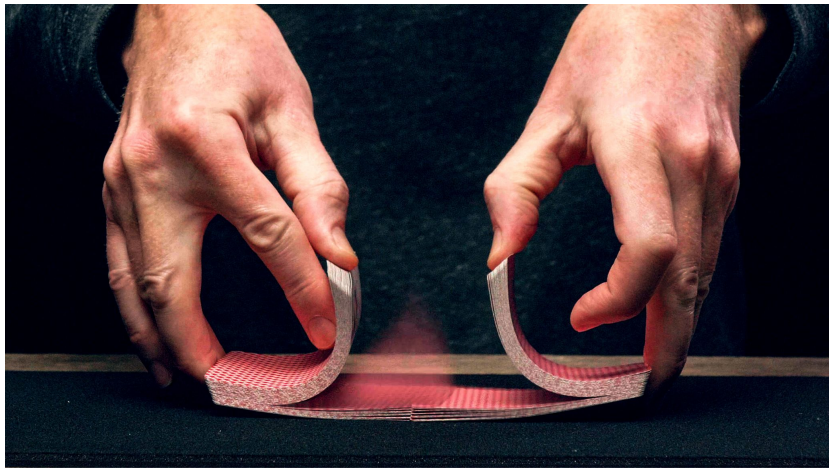
```
>>> nombre_repetition(L)
3

>>> nombre_repetition([i for i in range(52)])
8

>>> nombre_repetition([i for i in range(26)])
20

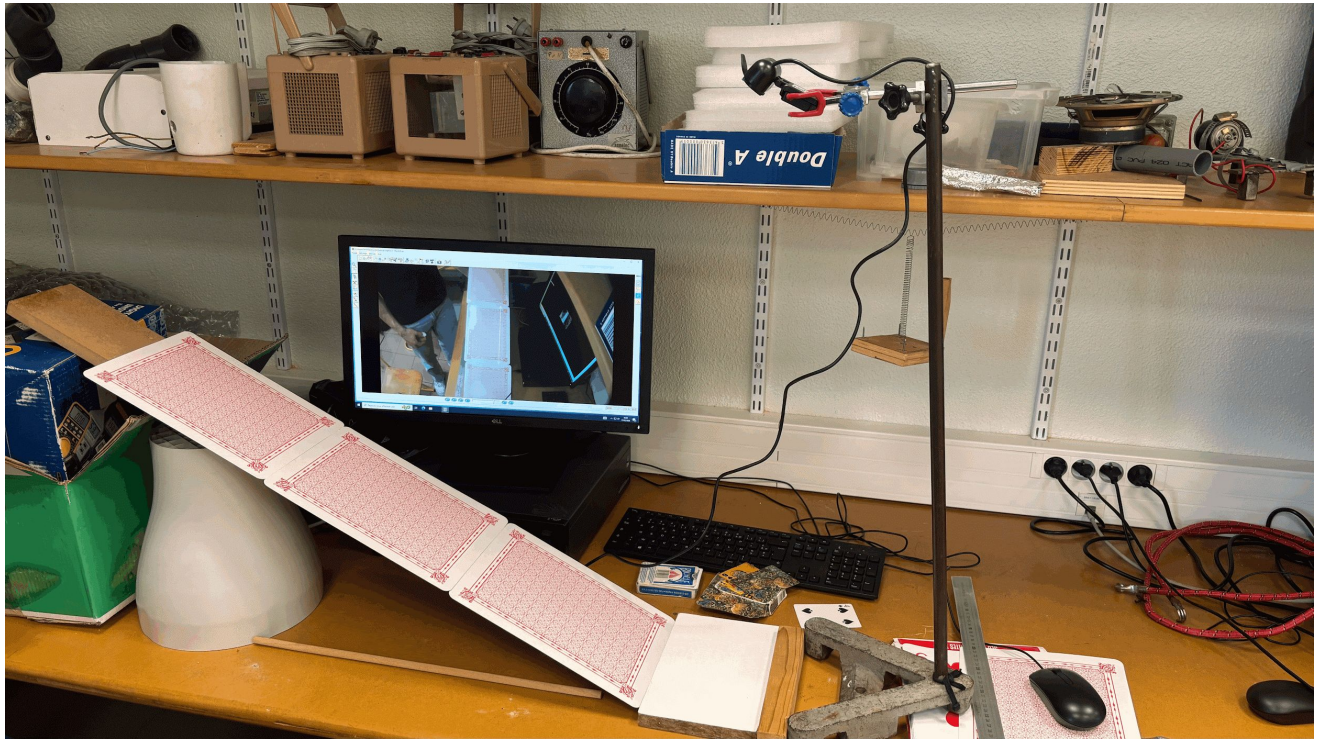
>>> nombre_repetition([i for i in range(54)])
52
```

2) *Etude du mélange réel*

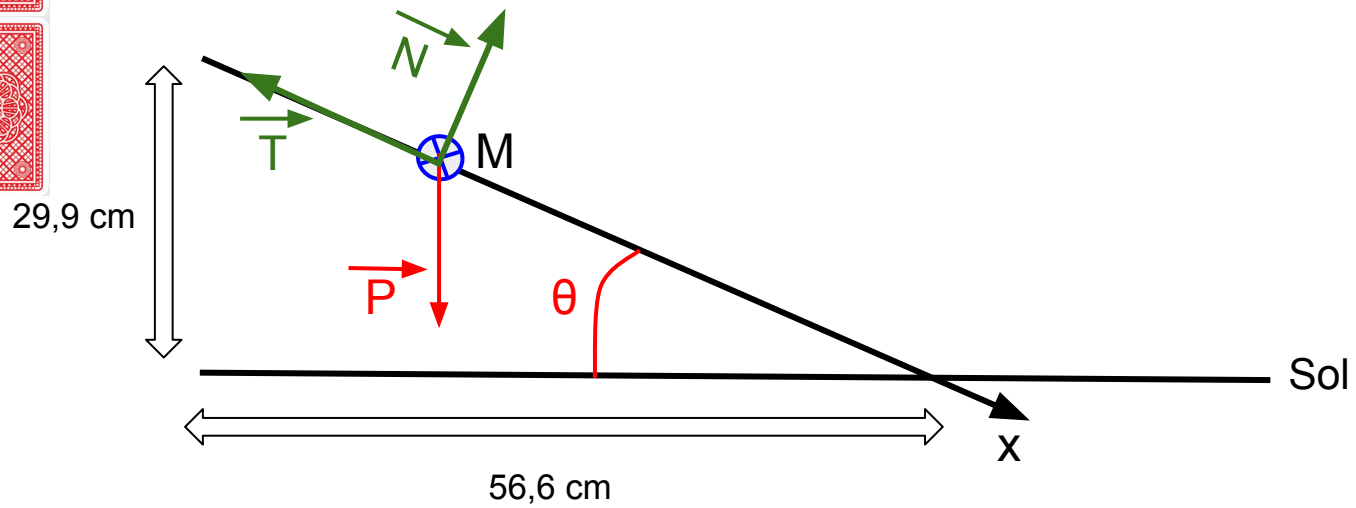


Dans la pratique, les cartes ne s'entrelacent pas parfaitement et deux ou trois cartes peuvent rester collées les unes sur les autres lors du mélange.

Acquisition du coefficient de frottement



Acquisition du coefficient de frottement



Système : M (m)
Référentiel : Terrestre (Galiléen)
Bilan des forces :

- $\vec{P} = mg(\cos(\theta)\vec{ey} - \sin(\theta)\vec{ex})$
- $\vec{T} = -T\vec{ex}$
- $\vec{N} = N\vec{ey}$

D’après les lois de Coulomb : T = f*N (en dynamique)

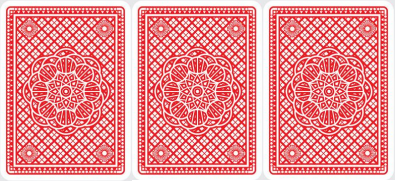
<u>On applique le Principe Fondamental de la Dynamique</u> :

$\overrightarrow{/ex}$: ma = mgsin(θ) - T T = mgsin(θ) - ma

$\overrightarrow{/ey}$: 0 = -mgcos(θ) + N N = mgcos(θ)

<u>D'où</u> :

f = tan(θ) - a/gcos(θ)





Après avoir réalisé 5 fois l'expérience sur une carte du jeu :

$\langle a \rangle_1 = 1{,}875 \text{ m/s}^2$

$\langle a \rangle_2 = 1{,}650 \text{ m/s}^2$

$\langle a \rangle_3 = 1{,}739 \text{ m/s}^2$

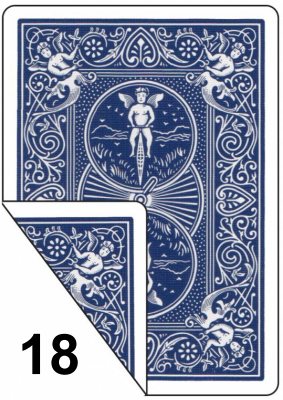
$\langle a \rangle_4 = 1{,}664 \text{ m/s}^2$

$\langle a \rangle_5 = 1{,}812 \text{ m/s}^2$

$\langle a \rangle_{tot} = 1{,}748 \text{ m/s}^2$

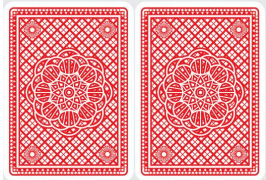
On obtient alors :

f = 0,327
u(f) = 0,086



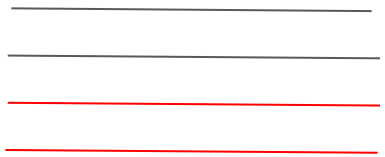
Il existe une adhérence entre les cartes

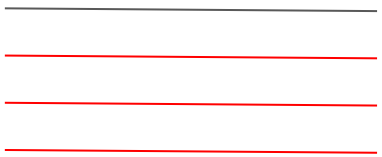


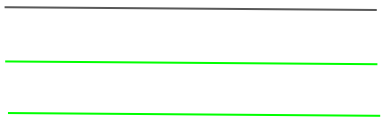


Utilisation des statistiques :

Pour jeu 1:
On a en moyenne : -k1=10 paquets de 2 cartes sur 52 cartes
-k2=4 paquets de 3 cartes sur 52 cartes







Paquet de 2 cartes

Paquet de 3 cartes

Espaces entre les cartes: intersections

1 intersection

2 intersections

Utilisation des statistiques :

Pour 52 cartes, il y a 51 intersections

On définit :
c1= k1/51= 0.196 et c2=k2/51=0.078

Probabilité d'avoir une intersection sélectionnée (deux cartes collées entre elles) = c1+c2
Probabilité d'avoir 3 cartes collées entre elles sachant que deux sont déjà collées = k2/(k1+k2)
= 1/(k1/k2+1) = 1/(c1/c2+1)

Utilisation des statistiques :

2 facteurs principaux :

1. Les cartes se collent entre-elles dans le paquet
2. Les cartes forment des paquets lors du mélange

On considère ces deux facteurs comme un seul car on a compté le nombre de paquet formé après 1 mélange.

Utilisation des frottements :

1) Les deux facteurs en un

carte_colle(L,c1,c2) :

```
>>> carte_colle([i for i in range(52)],0.196,0.078)
[[0], [1], [2], [3], [4], [5], [6], [7], [8], [9, 10], [11], [12], [13, 1
4], [15], [16], [17, 18], [19], [20], [21, 22, 23], [24], [25, 26], [27,
28], [29], [30, 31], [32], [33, 34, 35], [36], [37, 38], [39], [40], [41]
, [42, 43], [44], [45], [46, 47], [48, 49], [50], [51]]
```

```
>>> carte_colle(L,0.196,0.078)
[[0, 1], [2, 3], [4], [5], [6], [7]]

>>> carte_colle(L,0.196,0.078)
[[0], [1], [2], [3], [4, 5, 6], [7]]

>>> carte_colle(L,0.196,0.078)
[[0], [1], [2, 3], [4], [5], [6], [7]]
```

Utilisation des frottements :

1) Les deux facteurs en un

coupe_colle(L,c1,c2) :

```
>>> coupe_paire(L)
[0, 4, 1, 5, 2, 6, 3, 7]
```

```
>>> coupe_colle(L,0.196,0.078)
[0, 1, 4, 2, 5, 6, 3, 7]

>>> coupe_colle(L,0.196,0.078)
[0, 1, 4, 2, 5, 3, 6, 7]

>>> coupe_colle(L,0.196,0.078)
[0, 4, 5, 6, 1, 2, 7, 3]
```

```
>>> coupe_colle([i for i in range(52)],0.196,0.078)
[0, 26, 1, 2, 27, 3, 28, 29, 4, 30, 31, 32, 5, 33, 6, 34, 35, 36, 7, 8, 3
7, 9, 38, 39, 10, 11, 40, 41, 42, 12, 13, 43, 14, 44, 15, 45, 16, 17, 46,
47, 18, 48, 19, 20, 21, 49, 22, 50, 23, 51, 24, 25]
```

Utilisation des frottements :

1) Les deux facteurs en un

nmbr_rep(L,c1,c2)

```
>>> nmbr_rep([i for i in range(8)],0.196,0.078)
8971

>>> nmbr_rep([i for i in range(8)],0.196,0.078)
3791

>>> nmbr_rep([i for i in range(8)],0.196,0.078)
18153
```

```
>>> nmbr_rep([i for i in range(10)],0.196,0.078)
587585
```

Utilisation des frottements :

1) Les deux facteurs en un

```
>>> comparaison(L,0.196,0.264,0.078,0.03)
(5785.2, 4663.48)

>>> comparaison(L,0.196,0.264,0.078,0.03)
(4926.53, 4146.56)

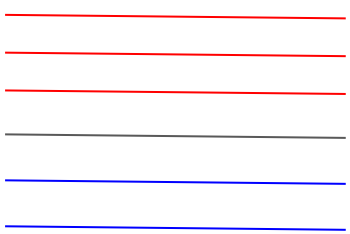
>>> comparaison(L,0.196,0.264,0.078,0.03)
(4678.19, 6440.29)
```

Utilisation des frottements :

2) Tentative d'ajouter un autre paramètre

calcul(0, k2, k3, n) renvoie le nombre de distribution du jeu de 2*n cartes avec k2 paquets de 2 cartes et k3 de 3 cartes

La distribution du jeu est réinitialisée avec la probabilité 1/calcul(0, k2, k3, n)





Utilisation des frottements :

2) Tentative d'ajouter un autre paramètre

coupe_colle2(L,c1,c2)

```
p=1-1/calcul(0,int(c1*n),int(c2*n),n)
```

```
>>> calcul(0,1,1,8)
156

>>> calcul(0,2,3,8)
560

>>> calcul(0,5,2,13)
408408
```

```
>>> nmbr_rep_final(L,0.196,0.078)
138

>>> nmbr_rep_final(L,0.196,0.078)
6047

>>> nmbr_rep_final(L,0.196,0.078)
15270

>>> nmbr_rep_final(L,0.196,0.078)
10999
```

3) *Conclusion sur le retour à l'état initial*

Notre méthode est instinctive et traduit explicitement le problème étudié.

Mais peut être optimisée.

Des phénomènes ont été négligés.

Annexes

Paquet de 2 cartes	Paquet de 3 cartes
8	2
16	2
19	1
7	3
12	0
16	1
14	1
13	0
11	3
18	0
12	2
14	2
12	2
15	1
15	3

Statistiques sur le nombre de cartes collées

Paquet 2	Paquet 3		Moyenne de paquets de 2	Moyenne de paquets de 3
7	8		10	4
8	6			
10	2			
8	3			
9	5			
10	2			
8	3			
11	3			
8	6			
9	5			
7	3			
7	4			
9	5			
8	5			
11	6			
14	3			
14	3			
10	6			
17	2			
14	4			
14	3			
11	7			

Annexes

```python
import random
import math

def coupe_paire(L):
    '''
    Simule et renvoie la coupe du jeu de carte associé à la
    liste L de cardinal pair
    '''
    assert len(L)%2==0
    M=[]
    n=len(L)
    P1,P2=L[0:int(n/2)],L[int(n/2):n]
    P=[(P1[i],P2[i]) for i in range(int(n/2))]
    for i in range (0,int(n/2)):
        M.append(P[i][0])
        M.append(P[i][1])
    return M


def coupe_impaire(L):
    '''
    Simule et renvoie la coupe du jeu de carte associé à la
    liste L de cardinal impair
    '''
    assert len(L)%2==1
    M=[]
    n=len(L)
    P1,P2=L[0:int(n/2)+1],L[int(n/2)+1:n]# le paquet du dessus à n+1 cartes
    P=[(P1[i],P2[i]) for i in range(int(n/2))]
    for i in range (0,int(n/2)):
        M.append(P[i][0])
        M.append(P[i][1])
    M.append(P1[int(n/2)])
    return M
```

Annexes

```
def nombre_repetition(L):
    '''
    L une liste d'entier
    Renvoie le nombre minimal de répétition en comparant à
    chaque instant le paquet mélangé au paquet initial
    '''
    P=coupe_paire(L)
    n=1
    while P!=L:
        n=n+1
        P=coupe_paire(P)
    return n

def cycle_disjoint(L):
    '''
    L une liste d'entier
    Renvoie la décomposition en cycle disjoint de la
    permutation obtenue lors du mélange de L
    '''
    M=coupe_paire(L)
    P=[]
    I=[]
    for i in range(1,len(L)+1):
        if i not in I:
            C=[i]
            I.append(i)
            k=M[i]
            while k!=i:
                I.append(k)
                C.append(k)
                k=M[k]
            if len(C)>1:
                P.append(C)
    return P
```

Annexes

```python
def ppcm(L):
    '''
    L une liste d'entier
    Renvoie le ppcm des longueurs des cycles dans la décomposition
    en cycle disjoint obtenu par le programme cycle_disjoint
    '''
    M = cycle_disjoint(L)
    n=len(M)
    ppcm=len(M[0])
    for i in range(1,n):
        ppcm=abs(ppcm * len(M[i])) // math.gcd(ppcm , len(M[i]))
        #pgcd(a,b)*ppcm(a,b)=|a*b|

    return ppcm
```

```python
def liste_liste(P):
    '''
    P une liste de listes
    Renvoie les éléments des listes de P en une seule liste,
    en conservant l'ordre
    '''
    L=[]
    for e in P:
        for i in range(len(e)):
            L.append(e[i])
    return L
```

Annexes

```
def carte_colle(L,c1,c2):
    '''
    L la liste et c1, c2 les probabilités d'avoir des paquets de 2
    et 3 cartes
    Créé une distribution de carte avec des cartes collées
    par paquets de 2 et de 3 cartes
    On parcourt la liste L (i est le numéro de la carte) et
    on détermine aléatoirement si la carte sélectionnée,
    se colle à celle du dessus.
    '''
    n=len(L)
    C=[]
    i=0
    while i+2<n-1:
        k=[L[i]]
        if  random.randint(0,1000)<1000*(c1+c2):
                #probabilité d'avoir un paquet de 2 cartes collées
            k.append(L[i+1])
            if  random.randint(0,1000)<1000*(1/(c1/c2+1)):
                #probabilité pour un paquet de 3 cartes collées
                k.append(L[i+2])
        i+=len(k)
        C.append(k)

    if  i+1<=n-1:
        if random.randint(0,1000)<1000*c1:
            C.append([L[i],L[i+1]])
            i=i+2
        else:
            C.append([L[i]])
            C.append([L[i+1]])
            i=i+2
    if i==n-1:
        C.append([L[i]])
    return C
```

Annexes

```
def coupe_colle(L,c1,c2):
    '''
    L la liste et c1, c2 les probabilitées d'avoir des paquets de 2
    et 3 cartes
    Coupe les deux paquets obtenus (séparation de L), après
    avoir considéré les paramètres aléatoire dans ces deux paquets
    '''
    assert len(L)%2==0
    k=[]
    P=[]
    M=[]
    n=len(L)
    P1,P2=L[0:n//2],L[n//2:n]
    P1= carte_colle(P1,c1,c2)
    P2= carte_colle(P2,c1,c2)
    n1=len(P1)
    n2=len(P2)
    if n1>n2:# le premier paquet à plus d'éléments que l'autre
        P=[(P1[i],P2[i]) for i in range(n2)]
        for e in P:
            M.append(e[0])
            M.append(e[1])
        P=M
        for i in range (n2,n1):
            P.append(P1[i])
    if n2>n1:# le deuxième paquet à plus d'éléments que l'autre
        P=[(P1[i],P2[i]) for i in range(n1)]
        for e in P:
            M.append(e[0])

            M.append(e[1])
        P=M
        for i in range (n1,n2):
            P.append(P2[i])
    if n1==n2:# les deux ont autant d'éléments
        P=[(P1[i],P2[i]) for i in range(n1)]
        for e in P:
            M.append(e[0])
            M.append(e[1])
        P=M
    K=liste_liste(P)
    return (K)
```

Annexes

```
def nmbr_rep(L,c1,c2):
    '''
    L la liste et c1, c2 les probabilités d'avoir des paquets de 2
    et 3 cartes
    Répète le programme coupe_colle tant que le paquet mélangé
    est différent du paquet
    initial
    '''
    assert len(L)%2==0
    i=0
    M=coupe_colle(L,c1,c2)
    while M!=L:
        i=i+1
        M=coupe_colle(M,c1,c2)
    return i

def calcul(p, k2, k3, n):
    """
    p: position de la dernière carte sélectionnée
    k2: nombre de paquets de 2 cartes (d'après les statistiques)
    K3: nombre de paquets de 3 cartes (d'après les statistiques)
    2*n-1: nombre d'écart entre les cartes
    Calcul le nombre de possibilité d'avoir K2 paquets de 2 cartes
    et K3 paquets de 3 cartes parmi 2n-p cartes, et sachant
    que les paquets sont espacés d'au moins 1 carte (afin de pouvoir
    différencier les paquets)
    On procède par double itération, sur p et K2 ou p et K3
    """
    if k2 == 0 and k3 == 0:
        return 1

    total = 0

    if k2 > 0:
        for i in range(p, 2*n - 1):
            total += calcul(i + 2, k2 - 1, k3, n)

    if k3 > 0:
        for i in range(p, 2*n - 2):
            total += calcul(i + 3, k2 , k3-1, n)

    return total
```

Annexes

```
def totale_possibilite(n):
    '''
    Pour un paquet de 2n cartes
    Renvoie le nombre total d'état possible du paquet de cartes
    '''
    totp=0
    for i in range(int(n+1)):
        for  j in range(int(2*n/3)+1):
            totp+=calcul(0, i, j, n)
    return (totp)

def proba(k2,k3,n):
    '''
    2n cartes, dont k2 cartes collées et K3 cartes collées deux fois
    Calcule de la probabilité d'avoir une combinaison
    correspondant aux statistiques
    '''
    return calcul(0, k2, k3, n)/totale_possibilite(n)

def liste_liste2(P):
    '''
    P est une liste de listes et d'entiers
    Renvoie les éléments des listes de P en une seule liste,
    en conservant l'ordre des termes
    '''
    L=[]
    for e in range(len(P)):
        if type(P[e])==int:
            L.append(P[e])
        else:
            for i in range(len(P[e])):
                L.append(P[e][i])
    return L
```

Annexes

```
def coupe_colle2(L,c1,c2):
    '''
    L la liste et c1, c2 les probabilités d'avoir des paquets de 2
    et 3 cartes
    Coupe les deux paquets obtenus (séparation de L), après
    avoir considéré les paramètres aléatoires dans ces deux paquets
    '''
    assert len(L)%2==0
    n=len(L)
    p=1-1/calcul(0,int(c1*n),int(c2*n),n)
    k=[]
    P=[]
    M=[]
    n=len(L)
    P1,P2=L[0:n//2],L[n//2:n]
    if random.randint(0,1000)<1000*p:
        P1= carte_colle(P1,c1,c2)
    if random.randint(0,1000)<1000*p:
        P2= carte_colle(P2,c1,c2)
```

Même programme que coupe_colle(L,c1,c2)

```
K=liste_liste2(P)
return (K)
```

Annexes

```
def nmbr_rep_final(L,c1,c2):
    '''
    L la liste et c1, c2 les probabilités d'avoir des paquets de 2
    et 3 cartes
    Répète le programme coupe_colle et coupe_colle2 tant que
    le paquet mélangé est différent du paquet initial
    '''
    assert len(L)%2==0
    i=0
    n=len(L)
    M=coupe_colle(L,c1,c2)
    while M!=L:
        i=i+1
        M=coupe_colle(M,c1,c2)
        M=coupe_colle2(M,c1,c2)
    return i

def comparaison(L,c1,c12,c2,c22):
    n=0
    m=0
    for i in range(100):
        n=n+ nmbr_rep(L,c1,c2)
        m=m+ nmbr_rep(L,c12,c22)
    return (n/100,m/100)
```